Rust from the Scratch

Draft edition – learn Rust from zero through small games and story chapters

Viacheslav Chub

Draft, 2026

Contents

Preface

You are about to learn **programming** — how to write instructions a computer follows. You will use **Rust**, a language that checks your work before anything runs. That sounds strict, but for a beginner it is helpful: Rust points to mistakes like a patient editor.

No prior coding experience is assumed. If you can read and follow steps, you can do this.

0.1 What programming is

Think of a recipe. It lists ingredients and steps: chop, stir, bake. A program is the same kind of thing — steps for the machine. The computer does not guess what you meant. It follows the recipe exactly.

Rust gives each ingredient a **type** (number, text, true/false) so the computer knows what fits where. That is one reason we chose Rust for this book: the rules are visible from day one.

0.2 Why Rust for your first language

Many courses start with a "easier" language and save strict typing for later. We start with Rust on purpose. Its rules match how careful thinking works:

- Names mean one thing at a time.
- Steps happen in order.
- Choices have clear branches.

You will not learn everything about Rust here. You **will** learn enough to build small games and read the screen output they produce.

0.3 Trust & Use

Rust sometimes shows syntax that looks mysterious — lines starting with `#`, words like `vec!`, or markers like `Some` and `None`.

Trust & Use means: copy those lines as written, run the program, watch the result, and move on. We explain them later in Trust & Use. You do not need to understand every symbol on first sight — the same way you can follow a map legend before you know how cartography works.

0.4 Three ways to read every idea

Throughout the book, each concept appears from three angles:

1. **Algorithm** — the steps the computer takes, in order.
2. **Types** — what kind of value each name holds (number, text, list, . . .).
3. **Language** — how the idea matches normal speech (a name tag, a fork in the road, a backpack of items).

You do not need three separate labels in your head. Just notice that a program is always **something stored**, **something done**, and **something said**.

0.5 How this book is organized

Each chapter is a **short novel** — a small scene or game. You build it in **Steps**. Every Step is a complete program you paste into the Rust Playground and run. There is no install until Chapter 11.

Chapters 1-2 introduce words and pictures. Chapters 3-6 add choices, lists, recipes (functions), and character sheets (structs). Chapters 7-10 are game novels: a map, a guessing game, tic-tac-toe, snake. Chapter 11 puts Snake on your own computer with real keyboard control.

Open CONTENTS.md for the full list.

0.6 Your first run

When you are ready, go to Chapter 1: First Words. You will print a name badge — your first program.

If a Step fails to compile, compare your code to the book one character at a time. Rust's error messages name the line — that is the compiler helping you.

Welcome. Let's write something that runs.

Chapter 1

First Words

1.1 Story hook

You arrive at a game convention. Volunteers hand out name badges printed by a small program. Today, **you** write that program.

1.2 What you will build

```
+------------------+
|  GAME CON 2026   |
|  Name: Mira      |
|  Level: 7        |
+------------------+
```

1.2.1 Step 1 — Say hello

Algorithm: run the program; print one line; stop.

Types: none yet — the message is baked into the program.

Language: like shouting a single sentence across the room.

```
// Playground
fn main() {
```

```
    println!("Hello!");
}
```

Expected output:

```
Hello!
```

Every Rust program starts with `fn main()`. That is where execution begins. `println!` prints a line and adds a newline at the end.

> **Trust & Use:** `println!` ends with `!` — it is a **macro**, a shorthand that expands into more code. Use it as shown; see Trust & Use.

1.2.2 Step 2 — Store a name

Algorithm: create a box labeled `name`, put text inside, print a greeting that uses the box.

Types: `name` holds text. In Rust we write that as `&str` (a read-only text slice).

Language: `name` is a label on a box; the box holds the word **Mira**.

```
// Playground
fn main() {
    let name = "Mira";
    println!("Hello, {}!", name);
}
```

Expected output:

```
Hello, Mira!
```

`let` creates a **variable** — a named place for a value. `{}` in the string is filled by `name`.

1.2.3 Step 3 — A score that can change

Algorithm: store name and level; print both; increase level; print again.

Types: `name` is text (`&str`); `level` is a whole number (`i32`).

Language: the level is a number on the badge you can update with a pen.

```
// Playground
fn main() {
    let name = "Mira";
    let mut level = 7;
    println!("{} is level {}", name, level);
    level = 8;
    println!("After one quest: level {}",
        level);
}
```

Expected output:

```
Mira is level 7
After one quest: level 8
```

`mut` means **mutable** — the value may change. Without `mut`, Rust would reject `level = 8`.

1.2.4 Step 4 — The full badge

Algorithm: print a border, then name, then level, then a closing border.

Types: same as Step 3 — text and integer.

Language: like filling in a paper form line by line.

```
// Playground
fn main() {
    let name = "Mira";
    let mut level = 7;

    println!("+------------------+");
    println!("|  GAME CON 2026   |");
    println!("|  Name: {}      |", name);
    println!("|  Level: {}       |", level);
    println!("+------------------+");
}
```

Expected output:

```
+------------------+
|  GAME CON 2026   |
|  Name: Mira      |
|  Level: 7        |
+------------------+
```

Change `name` and `level` to your own values and run again.

1.3 Full chapter solution

```rust
// Playground
fn main() {
    let name = "Mira";
    let level = 7;

    println!("+------------------+");
    println!("|  GAME CON 2026   |");
    println!("|  Name: {}      |", name);
    println!("|  Level: {}        |", level);
    println!("+------------------+");
}
```

1.4 See also

Next: Chapter 2 — Patterns on the Wall — draw pictures with loops.

Chapter 2

Patterns on the Wall

2.1 Story hook

Deep in the tutorial dungeon, a blank wall waits for decoration. You teach the computer to stamp symbols in rows and columns — your first **picture** made of text.

2.2 What you will build

A hollow square frame:

```
########
#......#
#......#
#......#
#......#
########
```

2.2.1 Step 1 — One row of dots

Algorithm: repeat printing `.` five times on one line.

Types: the loop counter is a whole number; `.` is a single **character** (`char`).

Language: like writing one sentence of five dots.

```
// Playground
fn main() {
    for _ in 0..5 {
        print!(".");
    }
    println!();
}
```

Expected output:

```
.....
```

`for _ in 0..5` runs the body **five times**. `0..5` means 0, 1, 2, 3, 4 — not including 5. We use `_` because we do not need the counter value. `print!` stays on the same line; `println!()` finishes the line.

> **Trust & Use:** `print!` also ends with `!` — same family as `println!`. See Trust & Use.

2.2.2 Step 2 — A solid rectangle

Algorithm: outer loop for rows; inner loop for columns; print `#` each time.

Types: row and column counters are numbers; `#` is `char`.

Language: reading a paragraph line by line, each line the same width.

```
// Playground
fn main() {
    for _ in 0..4 {
        for _ in 0..6 {
            print!("#");
        }
        println!();
    }
}
```

Expected output:

```
######
######
```

```
######
######
```

Four rows, six columns — a filled block.

2.2.3 Step 3 — A growing triangle

Algorithm: row `r` prints `r + 1` stars; repeat for four rows.

Types: `row` is `i32`; `*` is `char`.

Language: each line of a poem is one word longer than the last.

```
// Playground
fn main() {
    for row in 0..4 {
        for _ in 0..=row {
            print!("*");
        }
        println!();
    }
}
```

Expected output:

```
*
**
***
****
```

`0..=row` includes `row` itself — so row 0 prints one star, row 3 prints four.

2.2.4 Step 4 — Hollow square frame

Algorithm: for each row, print `#` at the sides and `.` in the middle; top and bottom rows are all `#`.

Types: `row` and `col` are numbers; cells are `char`.

Language: a picture frame — thick border, empty center.

```
// Playground
fn main() {
    let size = 6;
    for row in 0..size {
        for col in 0..size {
            let edge = row == 0 || row == size
                - 1 || col == 0 || col == size
                - 1;
            if edge {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}
```

Expected output:

```
######
#....#
#....#
#....#
#....#
######
```

`if edge` chooses the character. `||` means **or** — any true condition makes `edge` true.

2.3 Full chapter solution

```
// Playground
fn main() {
    let size = 6;
    for row in 0..size {
        for col in 0..size {
            let edge = row == 0 || row == size
```

```
                - 1 || col == 0 || col == size
                - 1;
            if edge {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}
```

2.4 See also

Next: Chapter 3 — Crossroads — choose paths and change the story.

Part I

Building Blocks

Chapter 3

Crossroads

3.1 Story hook

The forest path splits. A locked door, a dark cave, a sunny meadow — your choices rewrite the ending. The playground cannot read your keyboard yet, so we **script** your decisions like lines in a play.

3.2 What you will build

```
Scene 1: You stand at a crossroads.
You go left.
You find a rusty key!
Scene 2: The door is locked.
You use the key. The treasure is yours!
You win!
```

3.2.1 Step 1 — Is the door locked?

Algorithm: set a true/false flag; if true, print one message; otherwise print another.

Types: `locked` is a **boolean** — only `true` or `false`.

Language: a yes/no question with two possible answers.

```
// Playground
fn main() {
    let locked = true;
    if locked {
        println!("The door is locked.");
    } else {
        println!("The door swings open.");
    }
}
```

Expected output:

```
The door is locked.
```

Change `locked` to `false` and run again to see the other branch.

3.2.2 Step 2 — Two endings

Algorithm: pick a path string; compare it; print the matching ending.

Types: `path` is text (`&str`); comparison yields `bool`.

Language: "If you went left, then … otherwise …"

```
// Playground
fn main() {
    let path = "left";
    if path == "left" {
        println!("You meet a friendly fox.");
    } else {
        println!("You meet a sleeping bear.");
    }
}
```

Expected output:

```
You meet a friendly fox.
```

3.2.3 Step 3 — A script of three choices

Algorithm: walk through a list of preset choices; print what happens at each step.

Types: `choices` is a fixed array of text slices; `choice` is one `&str`.

Language: reading stage directions aloud, one line at a time.

```
// Playground
fn main() {
    let choices = ["left", "right", "left"];
    for choice in choices {
        println!("You go {}.", choice);
        if choice == "left" {
            println!("  A coin glints in the
                grass.");
        } else {
            println!("  Wind rushes through the
                trees.");
        }
    }
}
```

Expected output:

```
You go left.
  A coin glints in the grass.
You go right.
  Wind rushes through the trees.
You go left.
  A coin glints in the grass.
```

Imagine you picked those directions on a keyboard — the array is your recorded play session.

3.2.4 Step 4 — Mini adventure with health and key

Algorithm: start with health and no key; follow scripted moves; update state; print win or lose.

Types: `health` is `i32`; `has_key` is `bool`; choices are text.

Language: a short story where inventory and health change the ending.

```
// Playground
fn main() {
    let mut health = 3;
    let mut has_key = false;
    let choices = ["left", "right", "left"];

    println!("Scene 1: You stand at a
        crossroads.");

    for choice in choices {
        println!("You go {}.", choice);
        if choice == "left" {
            println!("You find a rusty key!");
            has_key = true;
        } else {
            println!("A thorn bush scratches
                you.");
            health -= 1;
        }
    }

    println!("Scene 2: The door is locked.");
    if has_key && health > 0 {
        println!("You use the key. The treasure
            is yours!");
        println!("You win!");
    } else if health <= 0 {
        println!("You are too tired to
            continue.");
        println!("You lose.");
    } else {
        println!("No key. You turn back.");
        println!("You lose.");
    }
}
```

Expected output:

```
Scene 1: You stand at a crossroads.
You go left.
You find a rusty key!
You go right.
A thorn bush scratches you.
You go left.
You find a rusty key!
Scene 2: The door is locked.
You use the key. The treasure is yours!
You win!
```

`&&` means **and** — both conditions must be true. `-=` subtracts from `health`.

3.3 Full chapter solution

Same as Step 4 above.

3.4 See also

Next: Chapter 4 — The Backpack — carry a list of items.

Chapter 4

The Backpack

4.1 Story hook

Every adventurer needs a backpack. You cannot see inside the computer's memory, but you **can** print a numbered list of what you carry — and check whether the key is there before you reach the door.

4.2 What you will build

```
Backpack (3 items):
  1. torch
  2. rope
  3. key
You have the key!
```

4.2.1 Step 1 — Empty pack

Algorithm: create an empty list; if length is zero, say so.

Types: `pack` is a `Vec<&str>` — a growable list of text items.

Language: an empty bag — nothing to show yet.

```
// Playground
fn main() {
    let pack: Vec<&str> = Vec::new();
    if pack.len() == 0 {
        println!("Your backpack is empty.");
    }
}
```

Expected output:

```
Your backpack is empty.
```

`Vec::new()` makes an empty vector. `len()` counts items.

4.2.2 Step 2 — Add three items

Algorithm: create empty list; push three names; print count.

Types: each item is `&str`; the list is `Vec<&str>`.

Language: dropping three objects into a bag, one at a time.

```
// Playground
fn main() {
    let mut pack: Vec<&str> = Vec::new();
    pack.push("torch");
    pack.push("rope");
    pack.push("key");
    println!("You carry {} items.", pack.len());
}
```

Expected output:

```
You carry 3 items.
```

`push` adds to the **end** of the list.

4.2.3 Step 3 — Numbered inventory

Algorithm: loop with index; print each item with its number.

Types: `i` is `usize` (unsigned size — good for counting); items are `&str`.

Language: reading a packing list: "1. torch, 2. rope, . . . "

```
// Playground
fn main() {
    let pack = vec!["torch", "rope", "key"];
    println!("Backpack ({} items):",
        pack.len());
    for i in 0..pack.len() {
        println!("  {}. {}", i + 1, pack[i]);
    }
}
```

Expected output:

```
Backpack (3 items):
  1. torch
  2. rope
  3. key
```

> **Trust & Use:** `vec!["torch", "rope", "key"]` builds a vector from a list you give. Copy it exactly. Details in Trust & Use.

4.2.4 Step 4 — Do I have the key?

Algorithm: build inventory; print it; check if `"key"` is in the list.

Types: `pack` is `Vec<&str>`; `contains` returns `bool`.

Language: patting your pockets before the locked door.

```
// Playground
fn main() {
    let pack = vec!["torch", "rope", "key"];
    println!("Backpack ({} items):",
        pack.len());
    for i in 0..pack.len() {
        println!("  {}. {}", i + 1, pack[i]);
    }
    if pack.contains(&"key") {
        println!("You have the key!");
```

```
    } else {
        println!("No key yet.");
    }
}
```

Expected output:

```
Backpack (3 items):
  1. torch
  2. rope
  3. key
You have the key!
```

Remove `"key"` from the list and run again to see the other message.

4.3 Full chapter solution

Same as Step 4 above.

4.4 See also

Next: Chapter 5 — Recipes — wrap repeated steps in functions.

Chapter 5

Recipes

5.1 Story hook

You drew walls in Chapter 2 and walked forest paths in Chapter 3. The same patterns appear again and again. A **function** is a named recipe — write it once, use it whenever you need that dish.

5.2 What you will build

A triangle from a recipe, then a tiny adventure scene built from two recipes:

```
*
**
***
****
--- adventure ---
You go left.
  A coin glints in the grass.
You go right.
  Wind rushes through the trees.
```

5.2.1 Step 1 — One-line recipe

Algorithm: define a function that prints; call it from `main`.

Types: the function returns nothing — Rust writes that as `()`.

Language: a card in the kitchen that says "say hello."

```
// Playground
fn greet() {
    println!("Welcome, traveler.");
}

fn main() {
    greet();
}
```

Expected output:

```
Welcome, traveler.
```

Functions can sit **above** `main`. Rust reads the whole file before running.

5.2.2 Step 2 — Recipe with a width parameter

Algorithm: pass a number; loop that many times; print dots.

Types: `width` is `i32`; the function still returns `()`.

Language: "draw a line this long" — the number is the instruction.

```
// Playground
fn dot_line(width: i32) {
    for _ in 0..width {
        print!(".");
    }
    println!();
}

fn main() {
    dot_line(5);
    dot_line(3);
}
```

Expected output:

```
.....
...
```

5.2.3 Step 3 — Recipe that returns a number

Algorithm: roll damage from a fixed "dice" value; return it; print in `main`.

Types: parameter and return are `i32`.

Language: a function is a question: "how much damage?" — answer is a number.

```
// Playground
fn roll_damage(base: i32) -> i32 {
    base + 2
}

fn main() {
    let hit = roll_damage(3);
    println!("You deal {} damage.", hit);
}
```

Expected output:

```
You deal 5 damage.
```

`-> i32` says the function **returns** an integer.

5.2.4 Step 4 — Triangle and adventure recipes together

Algorithm: `print_triangle` draws stars; `describe_move` prints one path outcome; `main` calls both.

Types: height is `i32`; choice is `&str`.

Language: two recipe cards — one for art, one for story.

```
// Playground
fn print_triangle(height: i32) {
    for row in 0..height {
```

```
        for _ in 0..=row {
            print!("*");
        }
        println!();
    }
}

fn describe_move(choice: &str) {
    println!("You go {}.", choice);
    if choice == "left" {
        println!("  A coin glints in the
            grass.");
    } else {
        println!("  Wind rushes through the
            trees.");
    }
}

fn main() {
    print_triangle(4);
    println!("--- adventure ---");
    describe_move("left");
    describe_move("right");
}
```

Expected output:

```
*
**
***
****
--- adventure ---
You go left.
  A coin glints in the grass.
You go right.
  Wind rushes through the trees.
```

5.3 Full chapter solution

Same as Step 4 above.

5.4 See also

Next: Chapter 6 — Character Sheet — group related fields in a struct.

Chapter 6

Character Sheet

6.1 Story hook

Before the boss fight, you open your character sheet: name, health, gold. In Rust, a **struct** is that sheet — one name for a bundle of fields that belong together.

6.2 What you will build

```
=== Character Sheet ===
Name:   Mira
Health: 8 / 10
Gold:   15
========================
After battle:
Health: 3 / 10
Gold:   20
```

6.2.1 Step 1 — One field

Algorithm: build a `Player` value; print the name field.

Types: `Player` is a struct type; `name` is `&str`.

Language: a form with one box filled in.

```
// Playground
struct Player {
    name: &'static str,
    health: i32,
    gold: i32,
}

fn main() {
    let hero = Player {
        name: "Mira",
        health: 10,
        gold: 15,
    };
    println!("Hero: {}", hero.name);
}
```

Expected output:

```
Hero: Mira
```

`struct` defines the **shape**; `{ name: "Mira", ... }` fills in the fields.

6.2.2 Step 2 — Print the full sheet

Algorithm: create hero; print formatted lines for each field.

Types: all fields stay as defined — text and two integers.

Language: reading a stat card out loud, line by line.

```
// Playground
struct Player {
    name: &'static str,
    health: i32,
    gold: i32,
}

fn main() {
    let hero = Player {
```

```
        name: "Mira",
        health: 10,
        gold: 15,
    };
    println!("=== Character Sheet ===");
    println!("Name:   {}", hero.name);
    println!("Health: {} / 10", hero.health);
    println!("Gold:   {}", hero.gold);
    println!("======================");
}
```

Expected output:

```
=== Character Sheet ===
Name:   Mira
Health: 10 / 10
Gold:   15
======================
```

6.2.3 Step 3 — Take damage and earn gold

Algorithm: make hero mutable; subtract health; add gold; print updates.

Types: `mut hero` allows field updates; fields keep their types.

Language: erasing and rewriting numbers on the paper sheet.

```
// Playground
struct Player {
    name: &'static str,
    health: i32,
    gold: i32,
}

fn main() {
    let mut hero = Player {
        name: "Mira",
        health: 10,
        gold: 15,
    };
```

```
    hero.health -= 2;
    hero.gold += 5;
    println!("Health: {}", hero.health);
    println!("Gold:   {}", hero.gold);
}
```

Expected output:

```
Health: 8
Gold:   20
```

Use `hero.health`, not `health` alone — the field lives **inside** the struct.

6.2.4 Step 4 — Tiny battle sequence

Algorithm: print sheet; apply scripted hits and rewards; print sheet again.

Types: same `Player` struct throughout.

Language: a combat log updating the same character card.

```
// Playground
struct Player {
    name: &'static str,
    health: i32,
    gold: i32,
}

fn print_sheet(hero: &Player) {
    println!("=== Character Sheet ===");
    println!("Name:   {}", hero.name);
    println!("Health: {} / 10", hero.health);
    println!("Gold:   {}", hero.gold);
    println!("======================");
}

fn main() {
    let mut hero = Player {
        name: "Mira",
        health: 10,
```

```
        gold: 15,
    };
    print_sheet(&hero);

    hero.health -= 7; // boss hit
    hero.gold += 5;   // loot

    println!("After battle:");
    print_sheet(&hero);
}
```

Expected output:

```
=== Character Sheet ===
Name:   Mira
Health: 10 / 10
Gold:   15
========================
After battle:
=== Character Sheet ===
Name:   Mira
Health: 3 / 10
Gold:   20
========================
```

`&hero` **borrows** the struct for printing without taking ownership.

> **Trust & Use:** The `&` means "look, do not take." Full rules in Trust & Use and Rust Core.

6.3 Full chapter solution

Same as Step 4 above.

6.4 See also

Next: Chapter 7 — Compass and Doors — move on a map with enums.

Part II

Game Novels

Chapter 7

Compass and Doors

7.1 Story hook

You stand on a tiny dungeon map. Walls block the way; doors open when you walk over them. Four compass directions drive each step — we script six moves and print the map after every turn.

7.2 What you will build

```
--- move 1 ---
.....
.@...
.....
.....
.....
--- move 2 ---
.....
..@..
.....
.....
.....
...
```

7.2.1 Step 1 — Name the four directions

Algorithm: define an enum with four variants; store one; print which way it is.

Types: `Direction` is an **enum** — a closed list of allowed values.

Language: compass points: north, south, east, west — only those words are valid.

```
// Playground
#[derive(Clone, Copy)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn main() {
    let step = Direction::East;
    match step {
        Direction::North => println!("Facing
            north."),
        Direction::South => println!("Facing
            south."),
        Direction::East => println!("Facing
            east."),
        Direction::West => println!("Facing
            west."),
    }
}
```

Expected output:

```
Facing east.
```

`match` picks the branch that fits the value — like a multiple-choice question with exact answers.

7.2.2 Step 2 — Turn direction into movement

Algorithm: given a direction, compute how `(x, y)` changes.

Types: position is `(i32, i32)` — a pair of integers; deltas are `(i32, i32)`.

Language: "east" means one step to the right on the grid.

```
// Playground
#[derive(Clone, Copy)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn delta(d: Direction) -> (i32, i32) {
    match d {
        Direction::North => (0, -1),
        Direction::South => (0, 1),
        Direction::East => (1, 0),
        Direction::West => (-1, 0),
    }
}

fn main() {
    let (dx, dy) = delta(Direction::East);
    println!("Move by ({}, {})", dx, dy);
}
```

Expected output:

```
Move by (1, 0)
```

7.2.3 Step 3 — Print the map once

Algorithm: build a 5x5 floor with walls on the border; place `@` at player position; print rows.

Types: `player` is `(i32, i32)`; each cell is `char`.

Language: a bird's-eye sketch on graph paper.

```
// Playground
fn print_map(player: (i32, i32), size: i32) {
    let (px, py) = player;
    for y in 0..size {
        for x in 0..size {
            let on_edge = x == 0 || y == 0 || x
                == size - 1 || y == size - 1;
            if x == px && y == py {
                print!("@");
            } else if on_edge {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn main() {
    print_map((2, 2), 5);
}
```

Expected output:

```
#####
#...#
#.@.#
#...#
#####
```

7.2.4 Step 4 — Scripted walk with six moves

Algorithm: start at center; for each direction in the script, add delta, clamp inside inner floor, print map.

Types: `moves` is an array of `Direction`; position stays `(i32, i32)`.

Language: watching a chess piece slide across the board move by move.

```
// Playground
#[derive(Clone, Copy)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn delta(d: Direction) -> (i32, i32) {
    match d {
        Direction::North => (0, -1),
        Direction::South => (0, 1),
        Direction::East => (1, 0),
        Direction::West => (-1, 0),
    }
}

fn print_map(player: (i32, i32), size: i32) {
    let (px, py) = player;
    for y in 0..size {
        for x in 0..size {
            let on_edge = x == 0 || y == 0 || x
                == size - 1 || y == size - 1;
            if x == px && y == py {
                print!("@");
            } else if on_edge {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn main() {
```

```
    let size = 5;
    let mut pos = (2, 2);
    let moves = [
        Direction::East,
        Direction::North,
        Direction::North,
        Direction::West,
        Direction::South,
        Direction::East,
    ];

    for (i, direction) in
        moves.iter().enumerate() {
        println!("--- move {} ---", i + 1);
        let (dx, dy) = delta(*direction);
        let nx = (pos.0 + dx).clamp(1, size -
            2);
        let ny = (pos.1 + dy).clamp(1, size -
            2);
        pos = (nx, ny);
        print_map(pos, size);
    }
}
```

Expected output:

```
--- move 1 ---
#####
#...#
#..@.#
#...#
#####
--- move 2 ---
#####
#.@.#
#...#
#...#
#####
--- move 3 ---
```

```
#####
#.@.#
#...#
#...#
#####
--- move 4 ---
#####
#...#
#.@.#
#...#
#####
--- move 5 ---
#####
#...#
#.@.#
#...#
#####
--- move 6 ---
#####
#...#
#..@.#
#...#
#####
```

`clamp` keeps the player off the wall cells. `.iter().enumerate()` gives move number and direction.

> **Trust & Use:** `#[derive(Clone, Copy)]` on `Direction` — see Trust & Use.

7.3 Full chapter solution

Same as Step 4 above.

7.4 See also

Next: Chapter 8 — Guess the Secret — loops and hot/cold bars.

Chapter 8

Guess the Secret

8.1 Story hook

A vault hides a secret number. You cannot type guesses in the playground yet, so the program tries a fixed list. A **bar chart** made of `#` and `.` shows how close each guess is — hot or cold, visible at a glance.

8.2 What you will build

```
Secret is 42
Guess 10 -> .......... (far)
Guess 40 -> ############ (close)
Guess 42 -> FOUND!
You win in 3 tries!
```

8.2.1 Step 1 — Rocket countdown

Algorithm: start at 3; while greater than zero, print and subtract; then blast off.

Types: `n` is `i32`.

Language: “3, 2, 1, lift-off” — same rhythm as a launch.

```
// Playground
fn main() {
    let mut n = 3;
    while n > 0 {
        println!("{}", n);
        n -= 1;
    }
    println!("Lift-off!");
}
```

Expected output:

```
3
2
1
Lift-off!
```

`while` repeats until the condition is false.

8.2.2 Step 2 — Search with fixed guesses

Algorithm: compare each guess to the secret; print hit or miss; stop early on success.

Types: all numbers are `i32`.

Language: checking numbered lockers until one opens.

```
// Playground
fn main() {
    let secret = 42;
    let guesses = [10, 40, 42, 99];
    for guess in guesses {
        if guess == secret {
            println!("Guess {} -> FOUND!",
                guess);
            break;
        } else {
            println!("Guess {} -> miss", guess);
        }
```

```
    }
}
```

Expected output:

```
Guess 10 -> miss
Guess 40 -> miss
Guess 42 -> FOUND!
```

`break` exits the loop early.

8.2.3 Step 3 — Hot/cold bar

Algorithm: map distance to bar length — closer guesses fill more # slots.

Types: `distance` is `i32`; bar is built as text.

Language: a thermometer filling up when you get warmer.

```
// Playground
fn bar_for(guess: i32, secret: i32) -> String {
    let distance = (guess - secret).abs();
    let slots = 12;
    let filled = (slots -
        distance.min(slots)).max(0);
    let hot = "#".repeat(filled as usize);
    let cold = ".".repeat((slots - filled) as
        usize);
    format!("{}{}", hot, cold)
}

fn main() {
    let secret = 42;
    println!("Guess 10 -> {}", bar_for(10,
        secret));
    println!("Guess 40 -> {}", bar_for(40,
        secret));
    println!("Guess 42 -> {}", bar_for(42,
        secret));
}
```

Expected output:

```
Guess 10 -> ..........
Guess 40 -> ##########..
Guess 42 -> ############
```

> **Trust & Use:** `.abs()`, `.repeat()`, and `format!` are standard helpers. Copy as shown; see Trust & Use.

8.2.4 Step 4 — Full scripted game

Algorithm: print secret hint; loop guesses with bars; count tries; celebrate win.

Types: `tries` is `i32`; optional win uses `Option` — either a number of tries or nothing yet.

Language: a game show host calling out each attempt.

```
// Playground
fn bar_for(guess: i32, secret: i32) -> String {
    let distance = (guess - secret).abs();
    let slots = 12;
    let filled = (slots -
        distance.min(slots)).max(0);
    let hot = "#".repeat(filled as usize);
    let cold = ".".repeat((slots - filled) as
        usize);
    format!("{}{}", hot, cold)
}

fn main() {
    let secret = 42;
    let guesses = [10, 40, 42];
    let mut tries = 0;
    let mut won: Option<i32> = None;

    println!("Secret is {}", secret);

    for guess in guesses {
```

```
        tries += 1;
        if guess == secret {
            println!("Guess {} -> FOUND!",
                guess);
            won = Some(tries);
            break;
        } else {
            println!("Guess {} -> {} (far)",
                guess, bar_for(guess, secret));
        }
    }

    match won {
        Some(n) => println!("You win in {}
            tries!", n),
        None => println!("Out of guesses."),
    }
}
```

Expected output:

```
Secret is 42
Guess 10 -> ..... (far)
Guess 40 -> ##########.. (far)
Guess 42 -> FOUND!
You win in 3 tries!
```

> **Trust & Use:** `Some(tries)` and `None` mean "a value exists" or "no value yet." We explain `Option` in Trust & Use.

8.3 Full chapter solution

Same as Step 4 above.

8.4 See also

Next: Chapter 9 — Tic-Tac-Toe Arena — a 3x3 board and win rules.

Chapter 9

Tic-Tac-Toe Arena

9.1 Story hook

Two players, one board, three in a row wins. You watch a full match unfold — move by move — like sitting ringside at a classic console game.

9.2 What you will build

```
--- move 1 (X at 0) ---
 X



--- move 5 (O at 4) ---
 X
 O
 X

...
Winner: X
```

9.2.1 Step 1 — Empty board

Algorithm: create nine cells; print as 3x3 grid with spaces for empty cells.

Types: `board` is `Vec<char>` — nine characters in row order.

Language: nine boxes on paper, still blank.

```
// Playground
fn print_board(board: &[char]) {
    for row in 0..3 {
        for col in 0..3 {
            let cell = board[row * 3 + col];
            if cell == ' ' {
                print!("   ");
            } else {
                print!(" {} ", cell);
            }
            if col < 2 {
                print!("|");
            }
        }
        println!();
        if row < 2 {
            println!("---+---+---");
        }
    }
}

fn main() {
    let board = vec![' '; 9];
    print_board(&board);
}
```

Expected output:

```
   |   |
---+---+---
   |   |
---+---+---
   |   |
```

Trust & Use: `vec![' '; 9]` makes nine copies of `' '`. See Trust & Use.

9.2.2 Step 2 — Place one mark

Algorithm: start empty; write `X` at index 0; print.

Types: index is `usize`; mark is `char`.

Language: crossing the first square with an X.

```
// Playground
fn print_board(board: &[char]) {
    for row in 0..3 {
        for col in 0..3 {
            let cell = board[row * 3 + col];
            if cell == ' ' {
                print!("   ");
            } else {
                print!(" {} ", cell);
            }
            if col < 2 {
                print!("|");
            }
        }
        println!();
        if row < 2 {
            println!("---+---+---");
        }
    }
}

fn main() {
    let mut board = vec![' '; 9];
    board[0] = 'X';
    print_board(&board);
}
```

Expected output:

```
 X |   |
---+---+---
   |   |
---+---+---
   |   |
```

9.2.3 Step 3 — Detect a row win

Algorithm: after each placement, check three rows, three columns, two diagonals.

Types: returns `Option<char>` — either the winning mark or nothing.

Language: scanning lines on the page for three matching letters.

```
// Playground
fn winner(board: &[char]) -> Option<char> {
    let lines = [
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8],
        [0, 3, 6],
        [1, 4, 7],
        [2, 5, 8],
        [0, 4, 8],
        [2, 4, 6],
    ];
    for line in lines {
        let a = board[line[0]];
        let b = board[line[1]];
        let c = board[line[2]];
        if a != ' ' && a == b && b == c {
            return Some(a);
        }
    }
    None
}

fn main() {
```

```
    let board = vec!['X', 'X', 'X', 'O', 'O', '
        ', ' ', ' ', ' '];
    match winner(&board) {
        Some(mark) => println!("Winner: {}",
            mark),
        None => println!("No winner yet."),
    }
}
```

Expected output:

```
Winner: X
```

> **Trust & Use:** `Some(a)` wraps a value; `None` means no winner. See Trust & Use.

9.2.4 Step 4 — Full scripted match

Algorithm: replay moves as `(index, mark)` pairs; print board after each; stop when someone wins.

Types: moves are `(usize, char)`; board is `Vec<char>`.

Language: a recorded game you replay move by move.

```
// Playground
fn print_board(board: &[char]) {
    for row in 0..3 {
        for col in 0..3 {
            let cell = board[row * 3 + col];
            if cell == ' ' {
                print!("   ");
            } else {
                print!(" {} ", cell);
            }
            if col < 2 {
                print!("|");
            }
        }
        println!();
```

```
        if row < 2 {
            println!("---+---+---");
        }
    }
}

fn winner(board: &[char]) -> Option<char> {
    let lines = [
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8],
        [0, 3, 6],
        [1, 4, 7],
        [2, 5, 8],
        [0, 4, 8],
        [2, 4, 6],
    ];
    for line in lines {
        let a = board[line[0]];
        let b = board[line[1]];
        let c = board[line[2]];
        if a != ' ' && a == b && b == c {
            return Some(a);
        }
    }
    None
}

fn main() {
    let mut board = vec![' '; 9];
    let moves = [(0, 'X'), (4, 'O'), (1, 'X'),
        (3, 'O'), (2, 'X')];

    for (i, (idx, mark)) in
        moves.iter().enumerate() {
        board[*idx] = *mark;
        println!("--- move {} ({}) at {} ---",
            i + 1, mark, idx);
```

```
        print_board(&board);
        if let Some(w) = winner(&board) {
            println!("Winner: {}", w);
            break;
        }
    }
}
```

Expected output:

```
--- move 1 (X) at 0 ---
 X |   |
---+---+---
   |   |
---+---+---
   |   |
--- move 2 (O) at 4 ---
 X |   |
---+---+---
   | O |
---+---+---
   |   |
--- move 3 (X) at 1 ---
 X | X |
---+---+---
   | O |
---+---+---
   |   |
--- move 4 (O) at 3 ---
 X | X |
---+---+---
 O | O |
---+---+---
   |   |
--- move 5 (X) at 2 ---
 X | X | X
---+---+---
 O | O |
---+---+---
```

```
   |   |
Winner: X
```

X takes top row — classic finish.

9.3 Full chapter solution

Same as Step 4 above.

9.4 See also

Next: Chapter 10 — Snake Trail — a moving snake on a grid.

Chapter 10

Snake Trail

10.1 Story hook

The snake slides across a grid — head first, body trailing. In the playground it follows a **fixed tape** of directions, like a flipbook animation. You watch frames print one after another until the snake eats the food.

10.2 What you will build

```
--- frame 1 ---
#####
#...#
#.@.#
#...#
#####
--- frame 4 ---
#####
#..@#
#.o.#
#..*#
#####
Game over: ate the food!
```

10.2.1 Step 1 — Static grid with head

Algorithm: draw bordered grid; place `@` at head position and `*` at food.

Types: head and food are `(i32, i32)` — pairs of coordinates.

Language: a comic panel with the hero and a star to collect.

```
// Playground
fn print_grid(head: (i32, i32), food: (i32,
    i32), size: i32) {
    let (hx, hy) = head;
    let (fx, fy) = food;
    for y in 0..size {
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            if x == hx && y == hy {
                print!("@");
            } else if x == fx && y == fy {
                print!("*");
            } else if wall {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn main() {
    print_grid((2, 2), (3, 3), 5);
}
```

Expected output:

```
#####
#...#
#.@*#
#...#
#####
```

10.2.2 Step 2 — One step to the right

Algorithm: move head one cell east; print before and after.

Types: coordinates are `i32`.

Language: one frame of a walking animation.

```
// Playground
fn print_grid(head: (i32, i32), food: (i32,
    i32), size: i32) {
    let (hx, hy) = head;
    let (fx, fy) = food;
    for y in 0..size {
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            if x == hx && y == hy {
                print!("@");
            } else if x == fx && y == fy {
                print!("*");
            } else if wall {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn main() {
    let food = (3, 3);
    let size = 5;
    let mut head = (2, 2);
    println!("--- before ---");
    print_grid(head, food, size);
    head.0 += 1;
    println!("--- after ---");
```

```
    print_grid(head, food, size);
}
```

Expected output:

```
--- before ---
#####
#...#
#.@*#
#...#
#####
--- after ---
#####
#...#
#..@*#
#...#
#####
```

10.2.3 Step 3 — Body follows the head

Algorithm: store body segments in a `Vec`; each step, push new head, pop tail unless food is eaten.

Types: `body` is `Vec<(i32, i32)>` — list of positions, head at front.

Language: a train of cars following the engine.

```
// Playground
#[derive(Debug, Copy, Clone)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn print_grid(body: &[(i32, i32)], food: (i32,
    i32), size: i32) {
    let (fx, fy) = food;
    for y in 0..size {
```

```
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            let is_head =
                body.first().map(|(hx, hy)| *hx
                == x && *hy == y) == Some(true);
            let is_body =
                body.iter().skip(1).any(|(bx,
                by)| *bx == x && *by == y);
            if is_head {
                print!("@");
            } else if is_body {
                print!("o");
            } else if x == fx && y == fy {
                print!("*");
            } else if wall {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn step(body: &mut Vec<(i32, i32)>, direction:
    Direction, food: (i32, i32)) -> bool {
    let (hx, hy) = body[0];
    let (nx, ny) = match direction {
        Direction::North => (hx, hy - 1),
        Direction::South => (hx, hy + 1),
        Direction::East => (hx + 1, hy),
        Direction::West => (hx - 1, hy),
    };
    body.insert(0, (nx, ny));
    let ate = (nx, ny) == food;
    if !ate {
        body.pop();
```

```
    }
    ate
}

fn main() {
    let size = 5;
    let food = (3, 3);
    let mut body = vec![(1, 2), (1, 3)];
    let dirs = [Direction::East,
        Direction::East, Direction::North];

    for (i, d) in dirs.iter().enumerate() {
        println!("--- frame {} ---", i + 1);
        let _ = step(&mut body, *d, food);
        print_grid(&body, food, size);
    }
}
```

Expected output:

```
--- frame 1 ---
#####
#...#
#o@.#
#..*#
#####
--- frame 2 ---
#####
#...#
#.o@#
#..*#
#####
--- frame 3 ---
#####
#..@#
#..o#
#..*#
#####
```

Trust & Use: `Copy` and `Clone` on the enum let you reuse

directions safely. See Trust & Use.

10.2.4 Step 4 — Full auto-play until food

Algorithm: run a direction tape; each step move head, grow on food, print frame; stop when food is eaten or wall is hit.

Types: `Direction` is copyable; body is `Vec<(i32, i32)>`.

Language: a short film of the snake reaching the star.

```
// Playground
#[derive(Clone, Copy, PartialEq)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn print_grid(body: &[(i32, i32)], food: (i32,
    i32), size: i32) {
    let (fx, fy) = food;
    for y in 0..size {
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            let is_head =
                body.first().map(|(hx, hy)| *hx
                == x && *hy == y) == Some(true);
            let is_body =
                body.iter().skip(1).any(|(bx,
                by)| *bx == x && *by == y);
            if is_head {
                print!("@");
            } else if is_body {
                print!("o");
            } else if x == fx && y == fy {
                print!("*");
```

```
            } else if wall {
                print!("#");
            } else {
                print!(".");
            }
        }
        println!();
    }
}

fn hit_wall(head: (i32, i32), size: i32) ->
    bool {
    let (x, y) = head;
    x <= 0 || y <= 0 || x >= size - 1 || y >=
        size - 1
}

fn main() {
    let size = 5;
    let food = (3, 3);
    let mut body = vec![(1, 3)];
    let tape = [
        Direction::North,
        Direction::East,
        Direction::East,
        Direction::South,
    ];

    for (i, dir) in tape.iter().enumerate() {
        let (hx, hy) = body[0];
        let next = match dir {
            Direction::North => (hx, hy - 1),
            Direction::South => (hx, hy + 1),
            Direction::East => (hx + 1, hy),
            Direction::West => (hx - 1, hy),
        };

        if hit_wall(next, size) {
```

```
            println!("--- frame {} ---", i + 1);
            println!("Game over: hit the
                wall!");
            break;
        }

        body.insert(0, next);
        if next == food {
            println!("--- frame {} ---", i + 1);
            print_grid(&body, (-1, -1), size);
            println!("Game over: ate the
                food!");
            break;
        }
        body.pop();

        println!("--- frame {} ---", i + 1);
        print_grid(&body, food, size);
    }
}
```

Expected output:

```
--- frame 1 ---
#####
#...#
#@..#
#..*#
#####
--- frame 2 ---
#####
#...#
#.@.#
#..*#
#####
--- frame 3 ---
#####
#...#
#..@#
```

```
#..*#
#####
--- frame 4 ---
#####
#...#
#..o#
#..@#
#####
Game over: ate the food!
```

> **Trust & Use:** `#[derive(Clone, Copy, PartialEq)]` on the enum — use as written; see Trust & Use.

10.3 Full chapter solution

Same as Step 4 above.

10.4 See also

Next: Chapter 11 — Live on Your Machine — install Rust and play Snake with the keyboard.

Part III

Your Machine

Chapter 11

Live on Your Machine

11.1 Story hook

You watched Snake follow a script in the playground. Now you install Rust on your computer and **drive** with the arrow keys. Same grid, same rules — real control.

11.2 What you will build

A terminal Snake game: arrow keys move the head, `q` quits, `@` is the head, `o` is the body, `*` is food.

Cargo only — this chapter needs files on disk and the `crossterm` crate.

11.2.1 Step 1 — Install Rust

Algorithm: download the installer, run it, open a new terminal, verify versions.

Types: no Rust code yet — shell commands only.

Language: setting up a workshop before building furniture.

Visit https://rustup.rs and follow the instructions for your system. On macOS or Linux, the one-line installer is:

```
curl --proto '=https' --tlsv1.2 -sSf
    https://sh.rustup.rs | sh
```

When it finishes, **close and reopen** your terminal, then verify:

```
rustc --version
cargo --version
```

Expected output (versions will differ):

```
rustc 1.87.0 (...)
cargo 1.87.0 (...)
```

If both commands print a version, you are ready.

11.2.2 Step 2 — Hello from Cargo

Algorithm: create a project folder, build, run the default program.

Types: Cargo manages the project; `main.rs` holds `fn main()`.

Language: naming a notebook and writing the first page.

```
cargo new snake_live
cd snake_live
cargo run
```

Open `src/main.rs` — it already contains a hello program. **Expected output:**

```
   Compiling snake_live v0.1.0 (...)
    Finished `dev` profile target(s) in ...
     Running `target/debug/snake_live`
Hello, world!
```

11.2.3 Step 3 — Add crossterm and draw the grid

Algorithm: add dependency; clear screen; print a bordered grid each frame.

Types: `Grid` holds size; coordinates stay `(i32, i32)`.

Language: preparing the stage before the actors enter.

Edit `Cargo.toml` — add under `[dependencies]`:

```
crossterm = "0.28"
```

Replace `src/main.rs` with:

```
// Cargo only
use crossterm::{
    cursor::{Hide, Show},
    execute,
    terminal::{Clear, ClearType},
};
use std::io::{self, Write};

fn print_grid(head: (i32, i32), food: (i32,
    i32), size: i32) -> io::Result<()> {
    let (hx, hy) = head;
    let (fx, fy) = food;
    for y in 0..size {
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            let ch = if x == hx && y == hy {
                '@'
            } else if x == fx && y == fy {
                '*'
            } else if wall {
                '#'
            } else {
                '.'
            };
            print!("{}", ch);
        }
        println!();
    }
    Ok(())
}

fn main() -> io::Result<()> {
```

```
    let mut stdout = io::stdout();
    execute!(stdout, Hide)?;
    execute!(stdout, Clear(ClearType::All))?;
    print_grid((2, 2), (3, 3), 8)?;
    execute!(stdout, Show)?;
    Ok(())
}
```

Run:

```
cargo run
```

Expected output: an 8x8 grid in the terminal with `@` near the center and `*` on the floor.

> **Trust & Use:** `?` after a fallible operation means "stop and report error if this failed." See Trust & Use.

11.2.4 Step 4 — Playable Snake

Algorithm: game loop — read key, update snake, detect food and walls, redraw until quit or game over.

Types: `body: Vec<(i32, i32)>` — same model as Chapter 10.

Language: the flipbook from Chapter 10, but you turn the pages with arrow keys.

Replace `src/main.rs` with the full game:

```
// Cargo only
use crossterm::{
    cursor::{Hide, MoveTo, Show},
    event::{self, Event, KeyCode, KeyEventKind},
    execute,
    terminal::{Clear, ClearType,
        EnterAlternateScreen,
        LeaveAlternateScreen},
};
use std::io::{self, Write};
use std::time::{Duration, Instant};
```

```
#[derive(Clone, Copy, PartialEq)]
enum Direction {
    North,
    South,
    East,
    West,
}

fn print_grid(body: &[(i32, i32)], food: (i32,
    i32), size: i32) -> io::Result<()> {
    let (fx, fy) = food;
    for y in 0..size {
        for x in 0..size {
            let wall = x == 0 || y == 0 || x ==
                size - 1 || y == size - 1;
            let is_head =
                body.first().map(|(hx, hy)| *hx
                == x && *hy == y) == Some(true);
            let is_body =
                body.iter().skip(1).any(|(bx,
                by)| *bx == x && *by == y);
            let ch = if is_head {
                '@'
            } else if is_body {
                'o'
            } else if x == fx && y == fy {
                '*'
            } else if wall {
                '#'
            } else {
                '.'
            };
            print!("{}", ch);
        }
        println!();
    }
    Ok(())
}
```

```
fn hit_wall(head: (i32, i32), size: i32) ->
    bool {
    let (x, y) = head;
    x <= 0 || y <= 0 || x >= size - 1 || y >=
        size - 1
}

fn hit_self(body: &[(i32, i32)]) -> bool {
    let (hx, hy) = body[0];
    body.iter().skip(1).any(|(x, y)| *x == hx
        && *y == hy)
}

fn main() -> io::Result<()> {
    let size = 12;
    let mut body = vec![(size / 2, size / 2)];
    let mut food = (size - 2, size / 2);
    let mut dir = Direction::East;
    let tick = Duration::from_millis(180);

    let mut stdout = io::stdout();
    execute!(stdout, EnterAlternateScreen,
        Hide)?;

    let mut last_tick = Instant::now();
    let mut game_over = false;
    let mut message = "Arrow keys move. Q
        quits.".to_string();

    loop {
        if
            event::poll(Duration::from_millis(50
            ))? {
            if let Event::Key(key) =
                event::read()? {
                if key.kind ==
                    KeyEventKind::Press {
                    match key.code {
```

```
                    KeyCode::Up => dir =
                        Direction::North,
                    KeyCode::Down => dir =
                        Direction::South,
                    KeyCode::Left => dir =
                        Direction::West,
                    KeyCode::Right => dir =
                        Direction::East,
                    KeyCode::Char('q') |
                        KeyCode::Char('Q')
                        => break,
                    _ => {}
                }
            }
        }
    }

    if last_tick.elapsed() >= tick &&
        !game_over {
        let (hx, hy) = body[0];
        let next = match dir {
            Direction::North => (hx, hy -
                1),
            Direction::South => (hx, hy +
                1),
            Direction::East => (hx + 1, hy),
            Direction::West => (hx - 1, hy),
        };

        if hit_wall(next, size) ||
            hit_self(&body) {
            game_over = true;
            message = "Game over! Press
                Q.".to_string();
        } else {
            body.insert(0, next);
            if next == food {
                food = (size - 2, (food.1 +
```

```
                        3) % (size - 2) + 1);
                    message = "Yum! Keep
                        going.".to_string();
                } else {
                    body.pop();
                }
            }
            last_tick = Instant::now();
        }

        execute!(stdout, MoveTo(0, 0),
            Clear(ClearType::All))?;
        print_grid(&body, food, size)?;
        println!();
        println!("{}", message);
        stdout.flush()?;
    }

    execute!(stdout, Show,
        LeaveAlternateScreen)?;
    Ok(())
}
```

Run:

```
cargo run
```

Expected behavior:

- A 12x12 grid appears.
- Arrow keys change direction; the snake moves every ~180 ms.
- Eating * grows the snake and moves food.
- Hitting # or your own body shows **Game over! Press Q.**
- **Q** returns to the normal terminal.

Compare this file to Chapter 10 — the grid drawing and body logic are the same ideas with keyboard and timing added.

11.3 Full chapter solution

Step 4 source is the complete game. Keep the project:

```
cd snake_live
cargo run
```

11.4 See also

- Trust & Use — macros, `?`, and `derive` explained
- Rust Core — Preface — the next book when you are ready for ownership and traits

Appendices

Appendix A

Playground Guide

A.1 When to use the playground

Use playground	Use your machine (Chapter 11+)
Chapters 1-10	Keyboard Snake, saving files
Learning syntax step by step	External crates (`crossterm`)
Sharing a link with a friend	Projects with many files

Playground: https://play.rust-lang.org/

A.2 How to run a snippet

1. Open the playground link above.
2. Select all default code and delete it.
3. Paste the snippet from the book (include `fn main()`).
4. Click **Run** (or press the run shortcut shown in the UI).
5. Read output in the panel on the right.

A.3 Rules (matches STYLE_GUIDE.md)

1. **One file** with `fn main()` — snippets marked **Playground** are complete programs.
2. **`std` only** — the playground cannot add dependencies from `Cargo.toml`.

3. **No keyboard input** — games use a preset list of moves. Imagine you pressed the keys; the program follows the script.
4. **Output via `println!`** — pictures are made from text characters.

A.4 Scripted input stand-in

When a game would read your keyboard, the book uses arrays like:

```
let choices = ["left", "right", "right"];
let moves = [Direction::Right, Direction::Down,
    Direction::Down];
```

The story tells you what each entry means. On your machine (Chapter 11), you replace the script with real keys.

A.5 Sharing code

1. Paste your working program into the editor.
2. Click **Share** to get a permalink.
3. Bookmark it if you want to return later.

A.6 Edition and channel

Book examples target **Edition 2021**, **stable** — same as the playground default.

A.7 When something fails

Problem	What to try
Red error text	Compare your code to the book character by character
No output	Make sure `fn main()` calls `println!`
“cannot find”	You may have deleted part of a Trust & Use line — copy it back

For local projects after Chapter 11:

```
cd your_project
cargo run
```

A.8 See also

- Trust & Use — syntax you copied without full explanation
- Chapter 11 — install Rust and run Snake locally

Appendix B

Trust & Use — Revealed

During the book you copied syntax and kept going. This appendix explains what those pieces mean. You do not need to memorize everything here — treat it as a reference you can reread after Chapter 11.

B.1 Macros (! at the end)

Names ending with `!` are **macros** — templates that expand into ordinary Rust code before the program runs.

Macro	What it does
`println!("Hi {}", name)`	Print a line with placeholders filled in
`print!(".")`	Print without a newline
`format!("{}{}", a, b)`	Build a `String` instead of printing
`vec![1, 2, 3]`	Create a `Vec` from a list of values
`vec![' '; 9]`	Create a vector with nine copies of one value

You used macros from day one. They save typing and reduce mistakes.

B.2 `#[derive(...)]`

Above a `struct` or `enum`, `#[derive(...)]` asks Rust to **auto-generate** common implementations.

Trait	Meaning for beginners
`Debug`	Lets you print a value for debugging with `{:?}`
`Clone`	`.clone()` makes a duplicate
`Copy`	Simple values copy automatically instead of moving (integers, coordinates, enums with `Copy`)
`PartialEq`	Lets you compare with `==`

Example from Chapter 10:

```
#[derive(Clone, Copy, PartialEq)]
enum Direction {
    North,
    South,
    East,
    West,
}
```

Without `Copy`, you would pass directions differently. For small game types, `Copy` keeps code simple.

B.3 `Option`: `Some` and `None`

`Option<T>` means **maybe a T, maybe nothing**.

Value	Meaning
`Some(42)`	A value is present
`None`	No value

Used in Chapter 8 for win tracking and Chapter 9 for winner detection:

```rust
match winner(&board) {
    Some(mark) => println!("Winner: {}", mark),
    None => println!("No winner yet."),
}
```

`match` forces you to handle both cases — Rust will not let you forget `None`.

B.4 `Result` and `?`

`Result<T, E>` means **success (`Ok`) or failure (`Err`)**. File and terminal operations can fail; `Result` carries that information.

In Chapter 11:

```rust
fn main() -> io::Result<()> {
    execute!(stdout, Hide)?;
    // ...
    Ok(())
}
```

The `?` suffix means: if this operation failed, return the error from `main` immediately. Otherwise continue.

For small programs, that is cleaner than nested `if let Err(...)`.

B.5 Borrowing with `&`

In Chapter 6 you saw:

```rust
fn print_sheet(hero: &Player) {
    println!("Name: {}", hero.name);
}
```

`&Player` means **borrow for reading** — the function looks at the sheet without taking it away. Rust Core — Chapter 1 explains ownership and borrowing in full.

B.6 Helper methods you met

Method	Role
`.abs()`	Absolute value of a number
`.repeat(n)`	Repeat a string or character `n` times
`.contains(&item)`	Check if a vector holds a value
`.clamp(min, max)`	Keep a number inside a range
`.len()`	Count items in a vector or string
`.push(x)`	Add to the end of a vector
`.pop()`	Remove from the end of a vector
`.insert(0, x)`	Insert at the front (Snake head)

These are ordinary functions tied to a type — Rust calls them **methods**.

B.7 What to learn next

You now know variables, control flow, functions, structs, enums, vectors, and small games. The next layer — **ownership**, **traits**, **errors in depth**, and **concurrency** — lives in **Rust Core**.

Read Rust Core Preface when you feel comfortable writing 50-100 lines of Rust on your own.

B.8 Quick map: where each idea appeared

Idea	First chapter
`println!`, `let`, `mut`	1 — First Words
`for`, `if`	2 — Patterns
`bool`, comparisons	3 — Crossroads
`Vec`, `vec!`	4 — Backpack
`fn`, parameters, return	5 — Recipes
`struct`	6 — Character Sheet
`enum`, `match`	7 — Compass
`while`, `Option`	8 — Guess

Idea	First chapter
2D grid, game rules	9 — Tic-Tac-Toe
Game loop, body `Vec`	10 — Snake Trail
`?`, `crossterm`, Cargo	11 — Live